

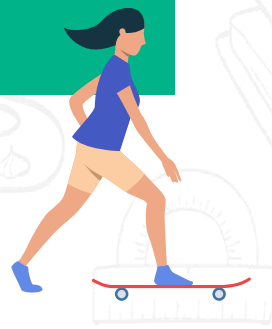
机器人操作系统（双语）

Robot Operating System (Bilingual)

WHERE THERE'S A WILL, THERE'S A WAY

授课教师：郄(qiè)龙飞

机器人工程



Overview lesson 9-1

1. Communication Model
2. C++ example-基于ROS编程
3. ~
4. ~

Objectives

1、掌握发布订阅的通讯模型

2、掌握 C++ 编写 ROS 程序的11个步骤

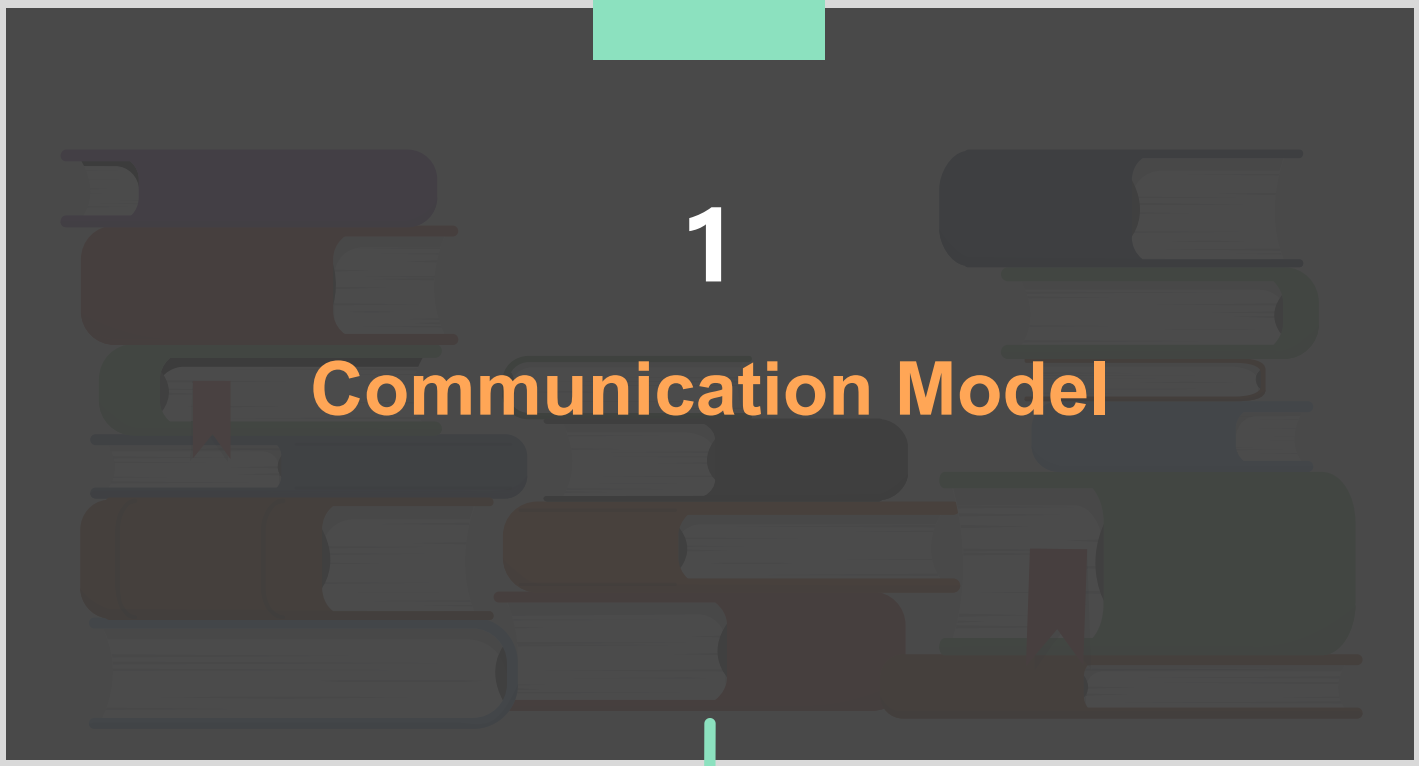


Pre-assessment

1、是否了解发布订阅模型?

2、是否用过 C++ 编写 ROS 程序?

3、是否了解 C++ 编写 ROS 程序的基本步骤?

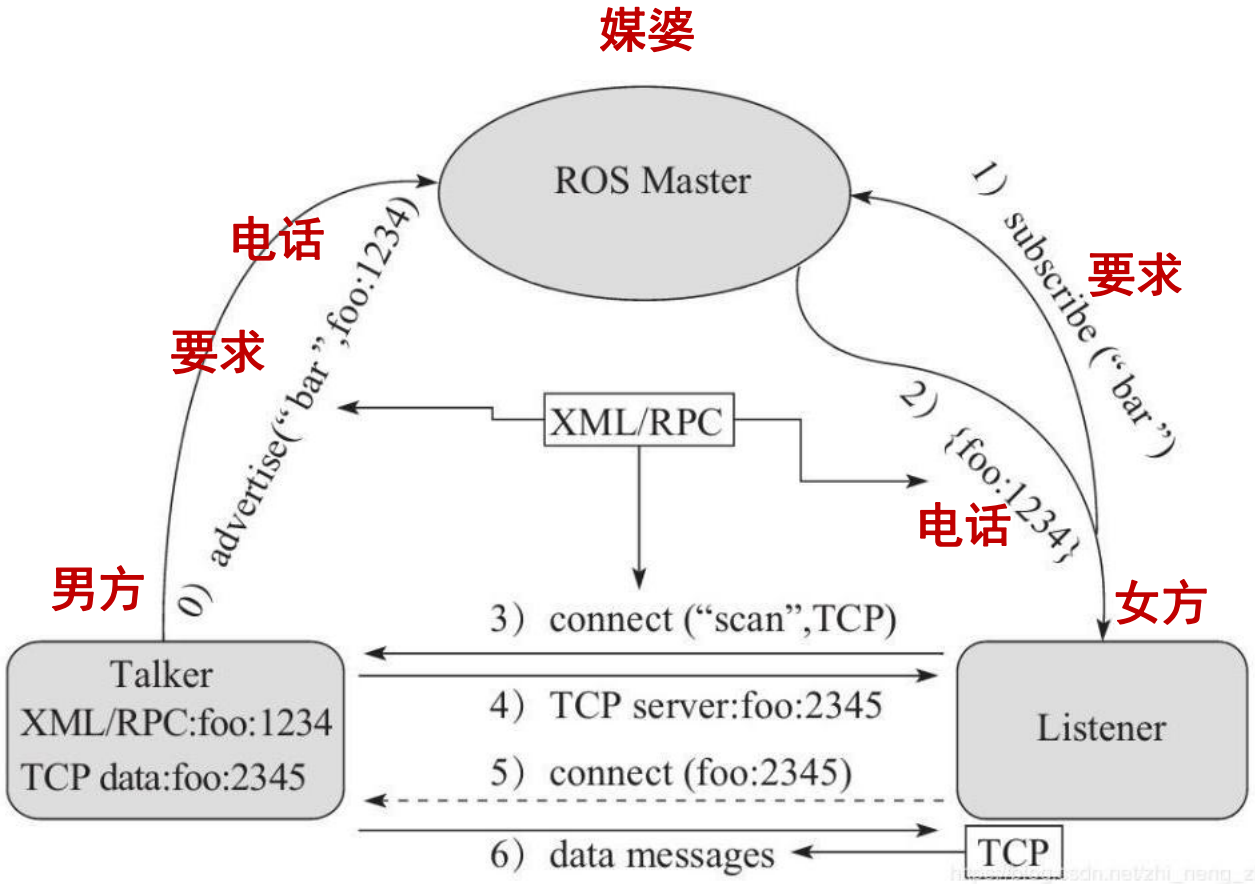


1

Communication Model

1.1 Topic model

1.1.1 Publish and subscribe



- 发布和订阅流程由以下步骤实现:

0.Talker 注册

Talker 启动后，会通过 RPC 在 ROS Master 中注册自身信息，其中包含所发布消息的话题名称。ROS Master 会将节点的注册信息加入到注册表中

1.Listener 注册

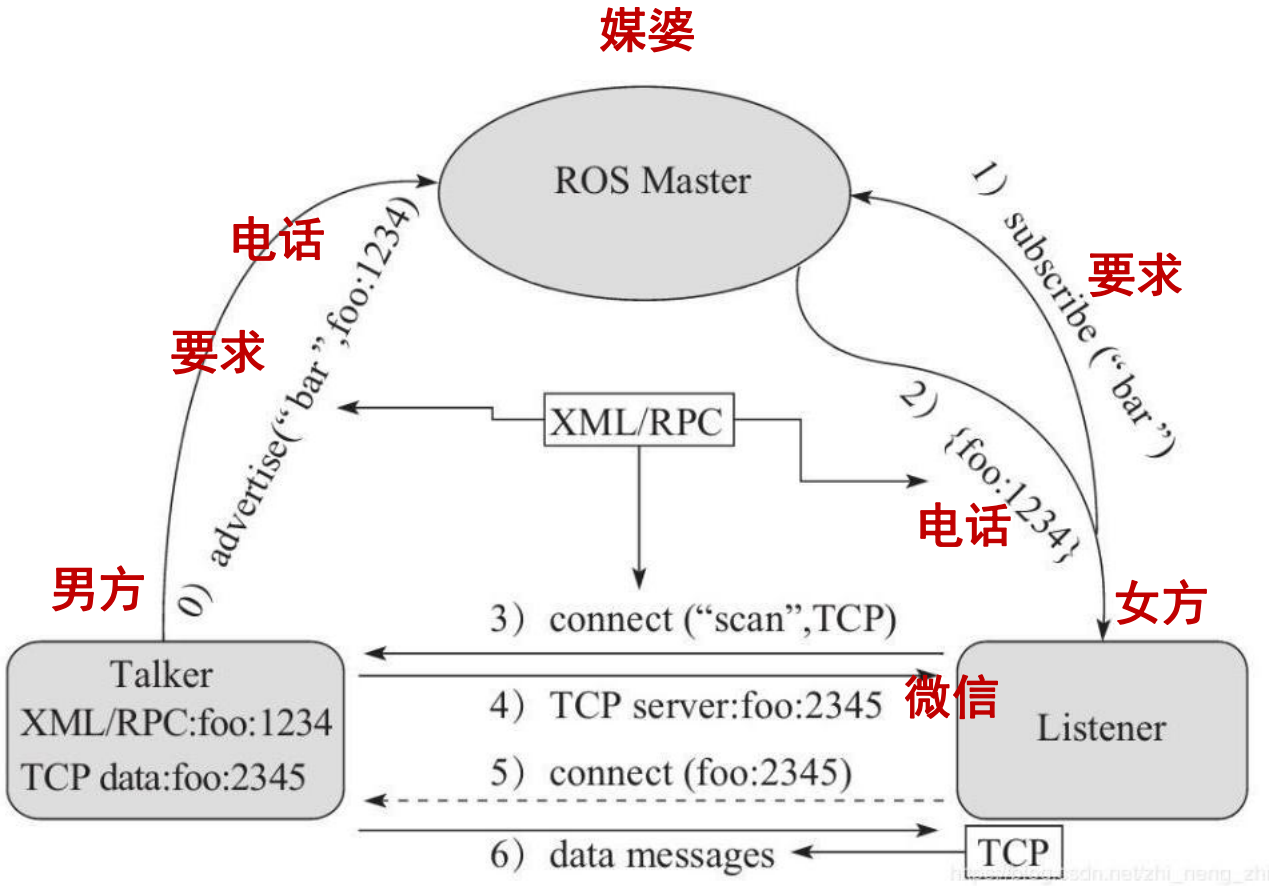
Listener 启动后，也会通过 RPC 在 ROS Master 中注册自身信息，包含需要订阅消息的话题名称。ROS Master 会将节点的注册信息加入到注册表中

2.ROS Master 实现信息匹配

ROS Master 会根据注册表中的信息匹配 Talker 和 Listener，并通过 RPC 向 Listener 发送 Talker 的 RPC 地址信息

1.1 Topic model

1.1.1 Publish and subscribe



3.Listener 向 Talker 发送请求

Listener 根据接收到的 RPC 地址，通过 RPC 向 Talker 发送连接请求，传输订阅的话题名称、消息类型以及通信协议(TCP/UDP)

4.Talker 确认请求

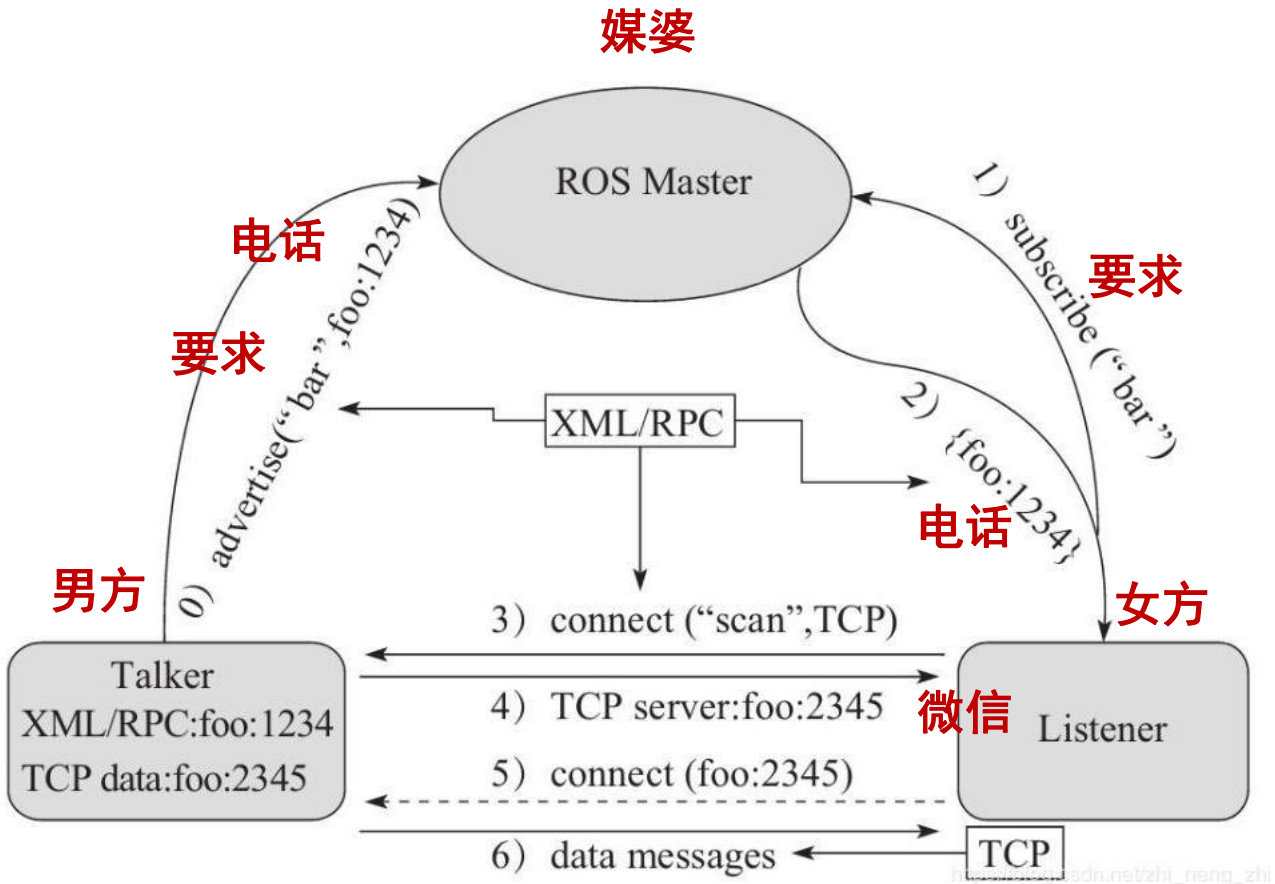
Talker 接收到 Listener 的请求后，通过 RPC 向 Listener 确认连接信息，并发送自身的 TCP 地址信息

5.Listener 与 Talker 建立连接

Listener 根据步骤4 返回的消息使用 TCP 与 Talker 建立网络连接

1.1 Topic model

1.1.1 Publish and subscribe



6.Talker向Listener发送消息

连接建立后，Talker 开始向 Listener 发布消息

注意1：上述实现流程中，前五步使用的 RPC 协议，最后两步使用的是 TCP 协议

注意2：Talker 与 Listener 的启动无先后顺序要求

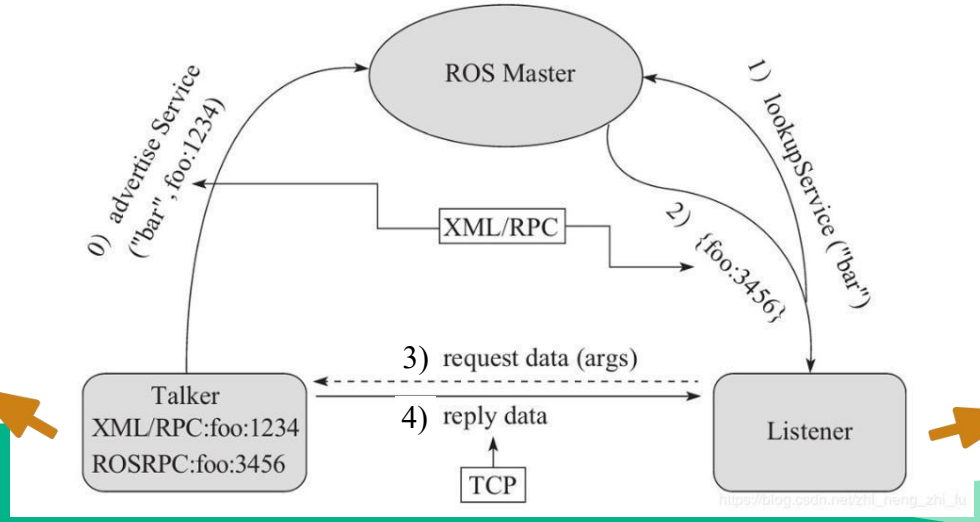
注意3：Talker 与 Listener 都可以有多个

注意4：Talker 与 Listener 连接建立后，不再需要 ROS Master。即便关闭 ROS Master，Talker 与 Listern 照常通信

1.2 Service model

1.2.1 Request and response

- 服务通信较之于话题通信更简单些，理论模型如下图所示，该模型中涉及到三个角色：
  1. ROS master(管理者)
  2. Server(服务端)
  3. Client(客户端)
- ROS Master 负责保管 Server 和 Client 注册的信息，并匹配话题相同的 Server 与 Client ，帮助 Server 与 Client 建立连接，连接建立后，Client 发送请求信息，Server 返回响应信息

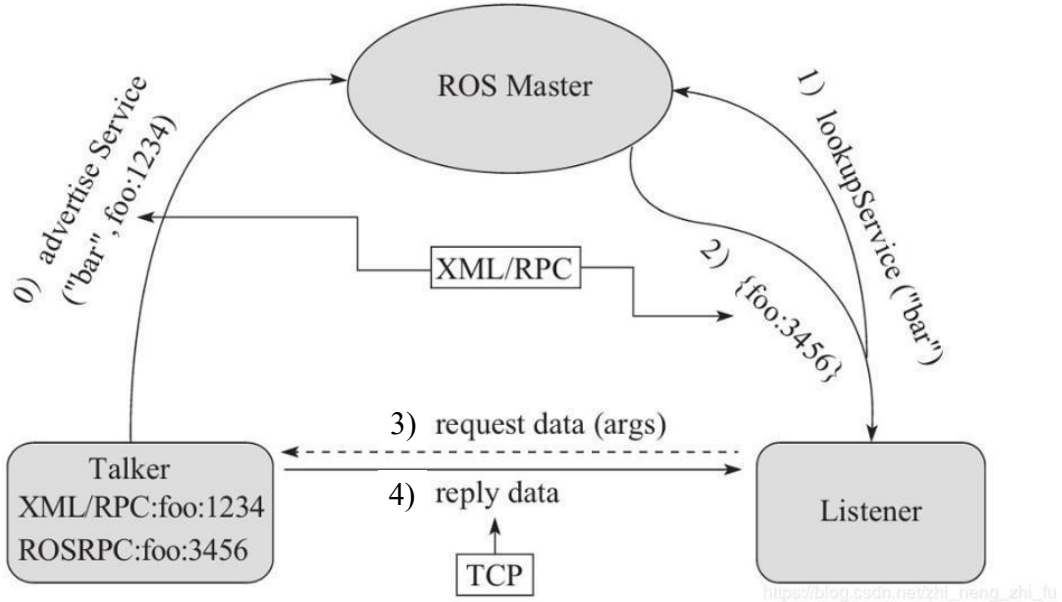


Server(服务端)

Client(客户端)

1.2 Service model

1.2.1 Request and response



注意:

1.客户端请求被处理时，需要保证服务器已经启动

2.服务端和客户端都可以存在多个

0.Server注册

Server 启动后，会通过 RPC 在 ROS Master 中注册自身信息，其中包含提供的服务的名称。ROS Master 会将节点的注册信息加入到注册表中

1.Client注册

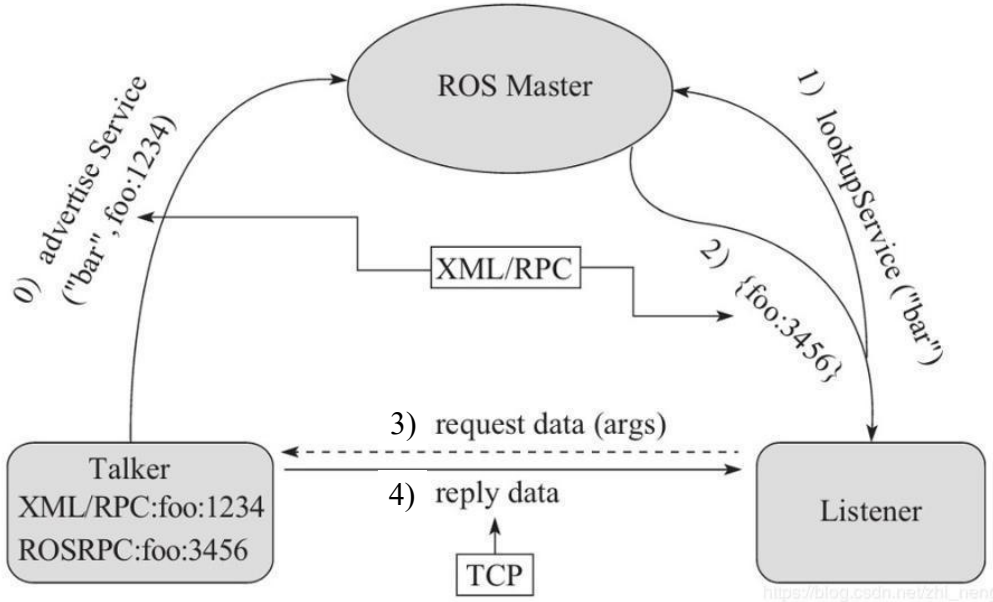
Client 启动后，也会通过 RPC 在 ROS Master 中注册自身信息，包含需要请求的服务的名称。ROS Master 会将节点的注册信息加入到注册表中

2.ROS Master实现信息匹配

ROS Master 会根据注册表中的信息匹配 Server 和 Client，并通过 RPC 向 Client 发送 Server 的 TCP 地址信息

1.2 Service model

1.2.1 Request and response



3.Client发送请求

Client 根据步骤2 响应的信息，使用 TCP 与 Server 建立网络连接，并发送请求数据

4.Server发送响应

Server 接收、解析请求的数据，并产生响应结果返回给 Client

注意:

1.客户端请求被处理时，需要保证服务器已经启动

2.服务端和客户端都可以存在多个

2

C++ example-基于 ROS 编程

2.1 ROS Programming

2.1.1 Overview

- ROS 编程是指通过使用 ROS 提供的一些内置函数来编写 ROS 应用程序
- ROS编译系统
  - ROS 有一套自己的用于编译 ROS 程序包的编译系统，该系统名称是 catkin (http://wiki.ros.org/catkin)
  - catkin 是由 CMake 编译系统和 Python 脚本定制的一套特殊的编译系统，以解决 ROS 程序包复杂的编译过程

2.1 ROS Programming

2.1.1 Overview

- ROS catkin 工作空间
  - src：存储新创建的功能包或者从远程代码库复制下来的功能。ROS功能包只有在 src 文件夹中才可以被创建、编译和执行
  - build：执行 catkin_make 命令时，catkin 工具会在 build 文件夹中创建一些中间编译文件和中间缓冲文件。这些缓冲文件可以避免在执行 catkin_make 命令时重新编译所有功能包
  - devel：执行 catkin_make 命令时，如果编译成功，创建的可执行文件存储在 devel 文件夹中，该文件夹中还有一些脚本，可以将该工作空间添加到 ROS 的系统环境变量中

2.1.2 使用ROS客户端库

- ROS客户端库是一系列实现了 ROS 概念所对应功能的代码
- 在编写 ROS 程序时，在代码中包含这些库函数便可以实现一个 ROS 节点
- 以下是主要的 ROS 客户端库：
  - roscpp：C++ 的 ROS 客户端库，被广泛用于 ROS 应用开发
  - rospy：Python 的 ROS 客户端库，其优点是语法简单，节省开发时间，但执行效率比 roscpp 低
  - roslisp：Lisp 语言的 ROS 客户端库
  - 其他客户端库：rosjava、rosnodejs、roslua

2.1.2 使用ROS客户端库

- 使用 roscpp 客户端库编写 ROS 节点：11个步骤(考试编程题)

1. 头文件和 ROS 模块

- 当用 C++ 编写代码时，第一部分要包含头文件
- 为了创建 ROS C++节点，需要包含以下头文件：

```
#include “ros/ros.h”
```

说明：ros.h 文件中包含了实现 ROS 功能所需的所有头文件

- 若要在节点中使用某种特定的消息类型，需要包含消息头文件
  - ROS 中提供一些内置的消息类型，用户也可以自定义消息类型
  - ROS 中的 std_msgs 内置消息功能包中包含：整型、浮点型、字符串型等标准数据类型的消息定义

2.1.2 使用ROS客户端库

1. 头文件和 ROS 模块

举例：要在代码中包含字符串消息，需要使用以下代码：

```
#include “std_msgs/Strings.h”
```

举例：若要使用自定义消息类型，需要使用以下代码：

```
#include “msg_pkg_name/message_name.h”
```

- std_msgs 功能包中的完整消息类型参见以下网址：
  http://wiki.ros.org/std_msgs

2.1.2 使用ROS客户端库

2. 初始化创建 ROS 节点

- 在开始编写任何 ROS 节点前，都需要首先初始化创建节点，此为必须要做的一步
- 在 C++ 中，使用以下代码初始化创建节点：

```
int main(int argc, char **argv)
  {
  ros::init(argc, argv, “name_of_node”)
  }
```

- 在 main() 函数中，必须包含 ros::init() 函数，该函数可以初始化创建 ROS 节点
- 可以向 init() 函数传递 argc, argv 命令行参数和节点名称

2.1.2 使用ROS客户端库

2. 初始化创建 ROS 节点

- int main(int argc, char **argv)（或者int main(int argc, char *argv[])）中：
  - 参数 argc 表示程序运行时发送给 main 函数的命令行参数的个数（包括可执行程序以及传参）
  - argv[]是字符指针数组，它的每个元素都是字符指针，指向命令行中每个参数的第一个字符
- 举例：假设已有一个myros的可执行文件，该文件支持传递参数

```
$ ./myros
```

此例中 argc=1， argv[0]=”myros”

```
$ ./myros myarg1 myarg2
```

此例中 argc=3， argv[0]=”myros”， argv[1]=”myarg1”， argv[2]=”myarg2”

2.1.2 使用ROS客户端库

3. 在 ROS 节点中打印消息

- ROS 提供了记录日志信息的应用程序接口，这些信息是一些可读的字符串，表示了节点的运行状态
- 在 C++ 中，以下函数可以用于记录节点信息：

1. ROS_INFO(string_msg, args)：记录节点输出的基本消息
2. ROS_WARN(string_msg, args)：记录节点输出的警告消息
3. ROS_DEBUG(string_msg, args)：记录节点输出的调试消息
4. ROS_ERROR(string_msg, args)：记录节点输出的错误消息
5. ROS_FATAL(string_msg, args)：记录节点输出的致命消息

2.1.2 使用ROS客户端库

4. 创建节点句柄

- 在初始化创建节点之后，需创建一个节点句柄实例，用于启动 ROS 节点和诸如发布、订阅话题等操作
- 在 C++ 中，使用以下命令创建节点句柄实例：

```
ros::NodeHandle nh
```

- 节点中的其他操作都可以通过使用 nh 实例来访问

2.1.2 使用ROS客户端库

5. 创建 ROS 消息定义

- 在发布话题前，需要创建一个 ROS 消息定义
- 在 C++ 中，使用以下代码创建 ROS 消息实例

```
std_msgs::String msg
```

- 创建了 ROS 消息实例后，可以使用以下命令向实例中添加数据

```
msg.data=”string data”
```

2.1.2 使用ROS客户端库

6. 在 ROS 节点中发布话题

- 在 C++ 中，使用以下语法在 ROS 节点中发布话题：

```
ros::Publisher publisher_object = node_handle.advertise<ROS message type>( “topic name” ,1000)
```

- 创建了一个待发布的话题后，publish() 命令能够通过话题来发送 ROS 消息：

```
publisher_object.publish(message)
```

2.1.2 使用ROS客户端库

6. 在 ROS 节点中发布话题

- 举例：

```
ros::Publisher chatter_pub = nh.advertise<std_msgs::String>(“chatter”,1000)

chatter_pub.publish(msg)
```

- 在上述例子中，chatter_pub 是 ROS 发布者的实例，chatter_pub 会发布消息类型为 std_msgs/String 的话题，话题的名称为 chatter，队列大小为 1000

2.1.2 使用ROS客户端库

7. 在 ROS 节点中订阅话题

- 发布话题时，需创建消息类型并且通过话题来发布该消息
- 订阅话题时，消息同样从话题被接收
- 在 C++ 中，使用以下命令订阅话题

```
ros::Subscriber subscriber_obj = node_handle.subscribe( “topic_name” ,1000,callback function)
```

- 在订阅话题时，不需要规定话题的消息类型，但必须给定话题名称和对应的回调函数
- 回调函数是用户自定义函数，用于响应话题。一旦从话题中接收到 ROS 消息，回调函数就会被执行
- 在回调函数中，可以操作 ROS 消息
- 举例：

```
ros::Subscriber sub = nh.subscribe( “chatter” ,1000,chatter_callback)
```

2.1.2 使用ROS客户端库

8. 在 ROS 节点中编写回调函数

- 当订阅 ROS 话题并且有消息到达话题时，回调函数被触发
- 在C++中，使用以下命令定义回调函数

```
void callback_name(const ros_message_const_pointer& pointer)
{
    //获取数据
    pointer-> data;
}
```

2.1.2 使用ROS客户端库

8. 在 ROS 节点中编写回调函数

- 举例：以下代码展示了如何在回调函数中处理 ROS 字符串消息并显示数据

```
void chatterCallback(const std_msgs::String::ConstPtr& msg)
{
    //获取数据
    ROS_INFO(“I heard: [%s]”, msg->data.c_str());
}
```

2.1.2 使用ROS客户端库

9. ROS 节点中的 ROS spin() 函数

- 在订阅或者发布函数之后，还需要调用一个函数来处理来自其他节点的订阅请求或发布请求
- 在 C++ 节点中，发布话题后需要调用 ros::spinOnce() 函数
- 订阅话题中，需要调用 ros::spin() 函数

2.1.2 使用ROS客户端库

10. ROS 节点中的 ROS sleep函数

- 若想为节点中的循环设置一个恒定的速率，可以使用 ros::Rate 功能
- 首先创建 ros::Rate 实例，并在实例中指定速率
- 创建实例后，调用 sleep() 函数，使得循环保持在指定的速率
- 举例：以下代码设置节点的循环速率为10次/s

```
ros::Rate r(10);  //10 Hz
r.sleep();
```

2.1.2 使用ROS客户端库

11. 设置和获取 ROS 参数

- 在 C++ 中，使用以下方法来获取参数：
  - 首先声明一个变量
  - 然后用节点句柄 node_handle 中的 getParam() 函数来获取所需要的参数

```
std::string global_name;
if(nh.getParam( “/global_name” ,global_name))
{
……
}
```

2.1.2 使用ROS客户端库

11. 设置和获取 ROS 参数

- 在 C++ 中，使用以下方法来设置参数：
  - 在 setParam() 函数中规定参数的名称和数值
- 举例：

```
nh.setParam(“/global_param”，5)
```

2.1 ROS Programming

2.1.2 使用ROS客户端库

作业：

1、熟练使用 Hello_world 中代码(编辑+调试)

2、更改代码中的输出参数，实现其他效果

```
robot@robot-pc:~$ rosrun hello_world talker
[ INFO] [1518291487.561201049]: hello world 0
[ INFO] [1518291487.662599248]: hello world 1
[ INFO] [1518291487.761577638]: hello world 2
[ INFO] [1518291487.862540060]: hello world 3
[ INFO] [1518291487.961661065]: hello world 4
[ INFO] [1518291488.061997374]: hello world 5
[ INFO] [1518291488.161413663]: hello world 6
[ INFO] [1518291488.261661379]: hello world 7
[ INFO] [1518291488.361571420]: hello world 8
[ INFO] [1518291488.461965544]: hello world 9
[ INFO] [1518291488.561633123]: hello world 10
[ INFO] [1518291488.661921779]: hello world 11
[ INFO] [1518291488.761608416]: hello world 12
```

```
robot@robot-pc:~$ rosrun hello_world listener
[ INFO] [1518291487.863502718]: I heard: [hello world 3]
[ INFO] [1518291487.961958201]: I heard: [hello world 4]
[ INFO] [1518291488.062417338]: I heard: [hello world 5]
[ INFO] [1518291488.161754224]: I heard: [hello world 6]
[ INFO] [1518291488.262099340]: I heard: [hello world 7]
[ INFO] [1518291488.361963797]: I heard: [hello world 8]
[ INFO] [1518291488.462384322]: I heard: [hello world 9]
[ INFO] [1518291488.561988817]: I heard: [hello world 10]
[ INFO] [1518291488.662334647]: I heard: [hello world 11]
[ INFO] [1518291488.761864631]: I heard: [hello world 12]
[ INFO] [1518291488.862494290]: I heard: [hello world 13]
[ INFO] [1518291488.961657606]: I heard: [hello world 14]
[ INFO] [1518291489.061816800]: I heard: [hello world 15]
[ INFO] [1518291489.161825396]: I heard: [hello world 16]
[ INFO] [1518291489.262011120]: I heard: [hello world 17]
[ INFO] [1518291489.361945019]: I heard: [hello world 18]
[ INFO] [1518291489.462888386]: I heard: [hello world 19]
[ INFO] [1518291489.562698356]: I heard: [hello world 20]
[ INFO] [1518291489.662093096]: I heard: [hello world 21]
[ INFO] [1518291489.761660963]: I heard: [hello world 22]
[ INFO] [1518291489.862041715]: I heard: [hello world 23]
[ INFO] [1518291489.961789634]: I heard: [hello world 24]
[ INFO] [1518291490.061721203]: I heard: [hello world 25]
[ INFO] [1518291490.161883229]: I heard: [hello world 26]
[ INFO] [1518291490.261879144]: I heard: [hello world 27]
[ INFO] [1518291490.361822851]: I heard: [hello world 28]
[ INFO] [1518291490.461525109]: I heard: [hello world 29]
[ INFO] [1518291490.561590742]: I heard: [hello world 30]
```

ROS Summary

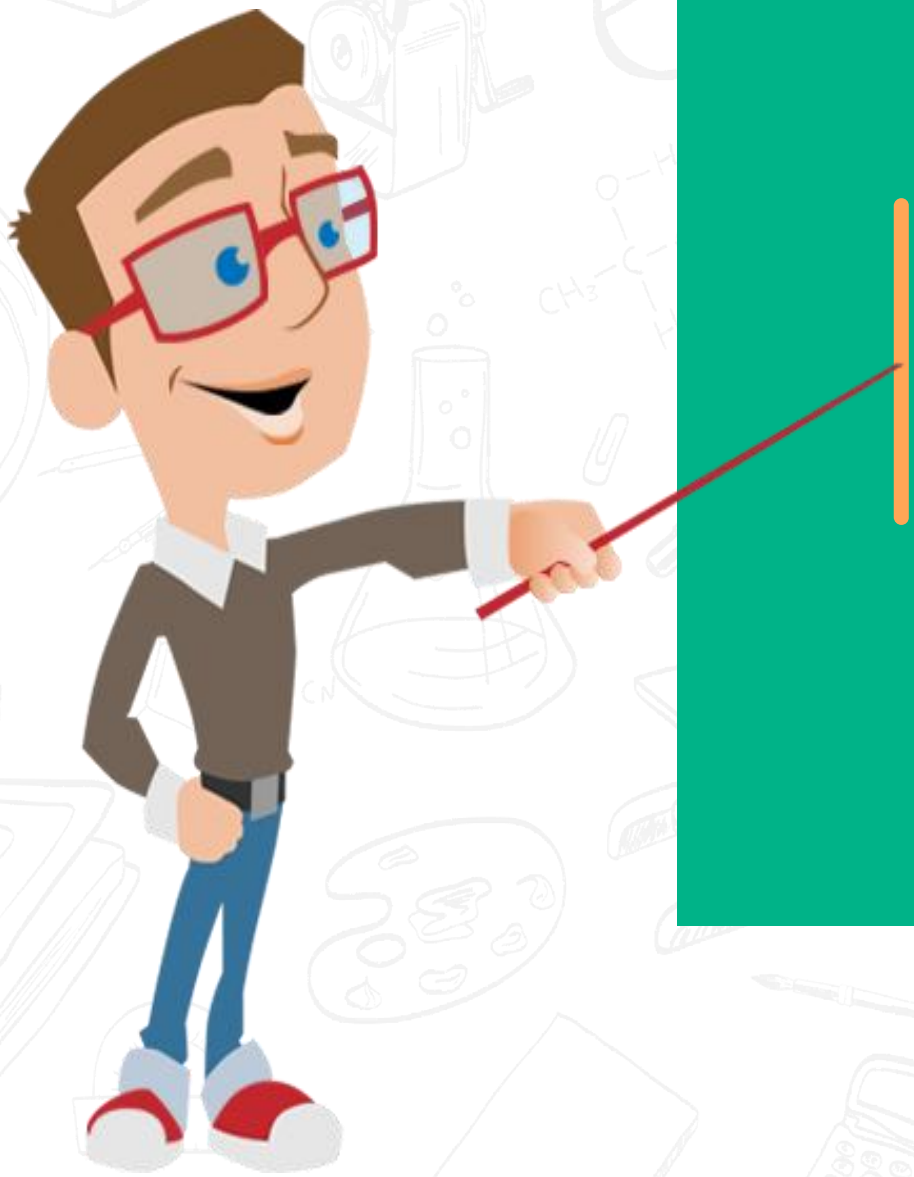
1. C++ example

    1. 头文件和 ROS 模块
    2. 初始化 ROS 节点
    3. 在 ROS 节点中打印消息
    4. 创建节点句柄
    5. 创建 ROS 消息定义
    6. 在 ROS 节点中发布话题
    7. 在 ROS 节点中订阅话题
    8. 在 ROS 节点中编写回调函数
    9. ROS 节点中的 ROS spin() 函数
    10. ROS节点中的 sleep 函数
    11. 设置和获取 ROS 参数

Q&A

今日事今日毕

Question?



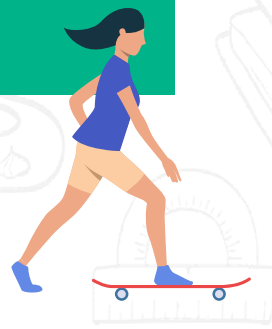
机器人操作系统（双语）

Robot Operating System (Bilingual)

WHERE THERE'S A WILL, THERE'S A WAY

授课教师：郄(qiè)龙飞

机器人工程



Overview lesson 9-2

1 VSCode Installation and Application

2 Topic Model C++ Code

3 ~

4 ~

Objectives

1、了解 VSCode 的使用和配置方法

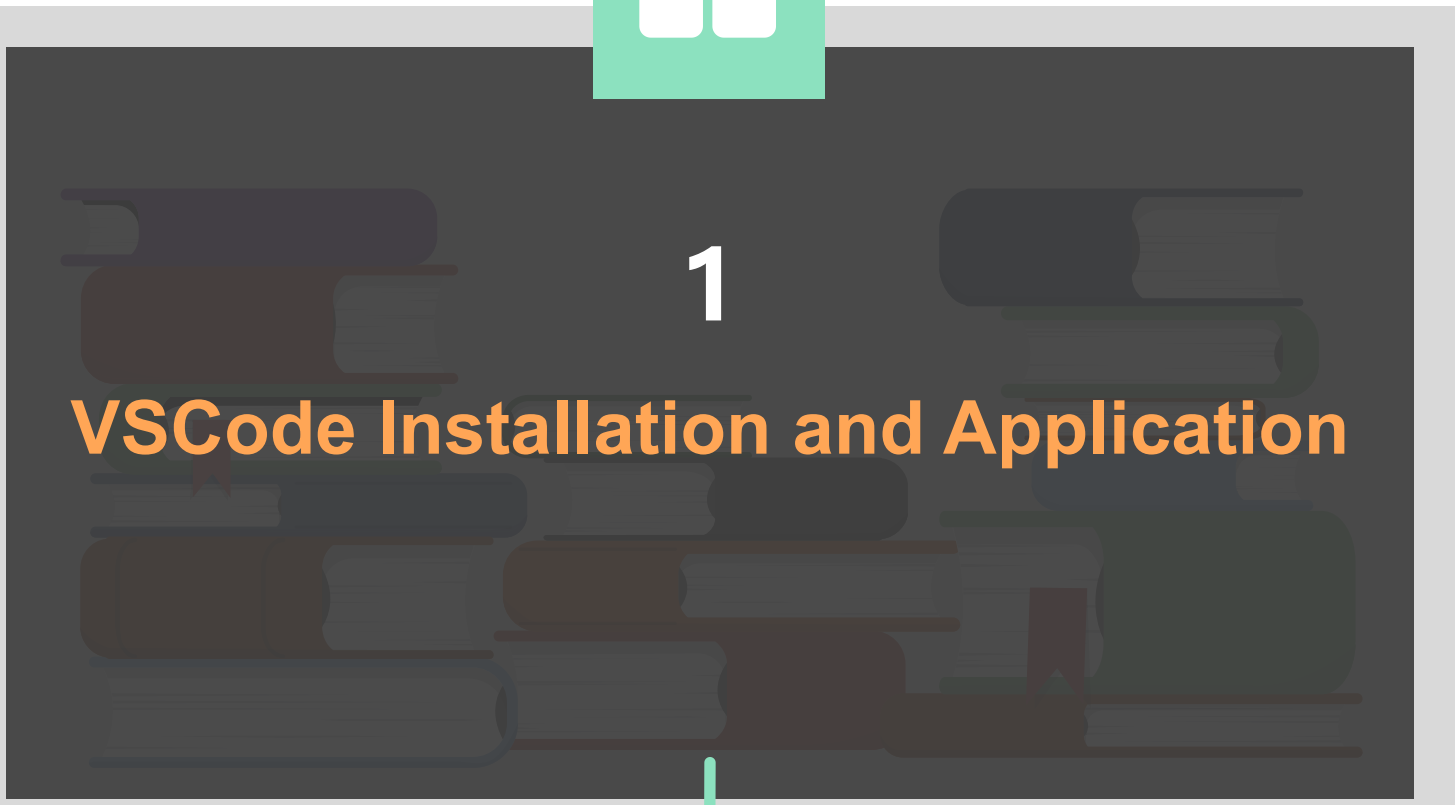
2、掌握 C++ 编写 ROS 发布和订阅节点程序



Pre-assessment

1、是否用过 VSCode?

2、是否成功运行 talker 和 listener?



1

VSCode Installation and Application

1.1 VSCode Introduction

1.1.1 Overview

- VSCode 全称 Visual Studio Code，是微软出的一款轻量级代码编辑器，免费、开源而且功能强大
- VSCode 支持几乎所有主流的程序语言的语法高亮、智能代码补全、自定义热键、括号匹配、代码片段、代码对比 Diff、GIT 等特性，支持插件扩展，并针对网页开发和云端应用开发做了优化
- 软件跨平台支持 Win、Mac 以及 Linux
- VSCode 下载：
  - VSCode 下载：https://code.visualstudio.com/docs?start=true
  - 历史版本下载链接：https://code.visualstudio.com/updates

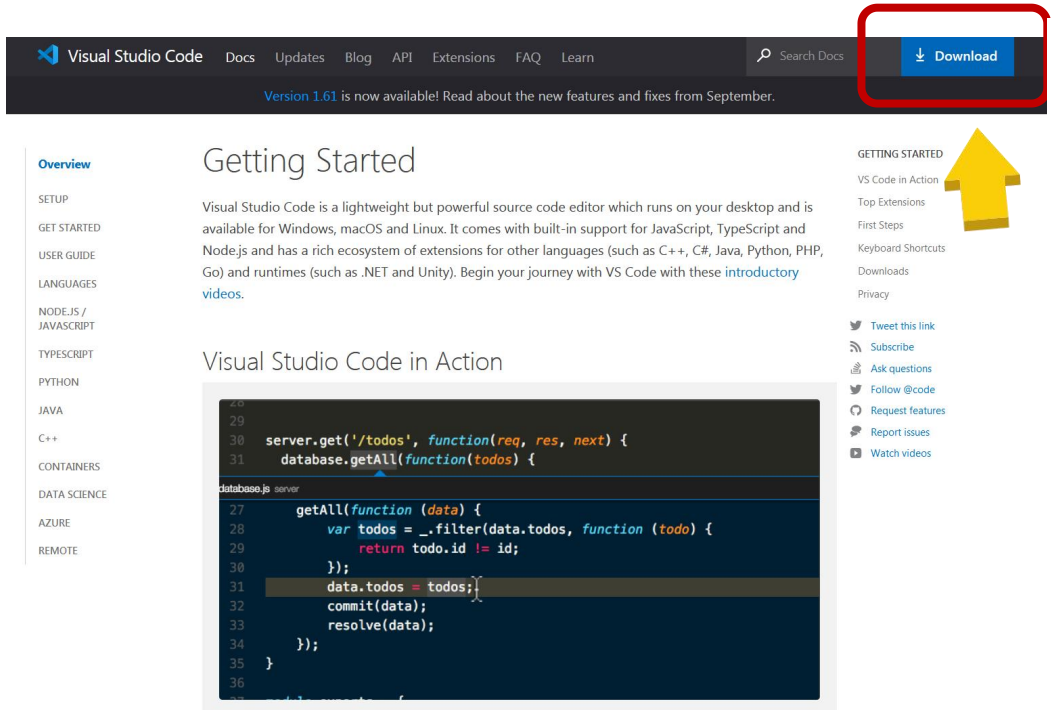
参考视频: https://www.bilibili.com/video/BV1Ci4y1L7ZZ?p=23
【奥特学园】ROS机器人入门课程《ROS理论与实践》零基础教程：23-32小节

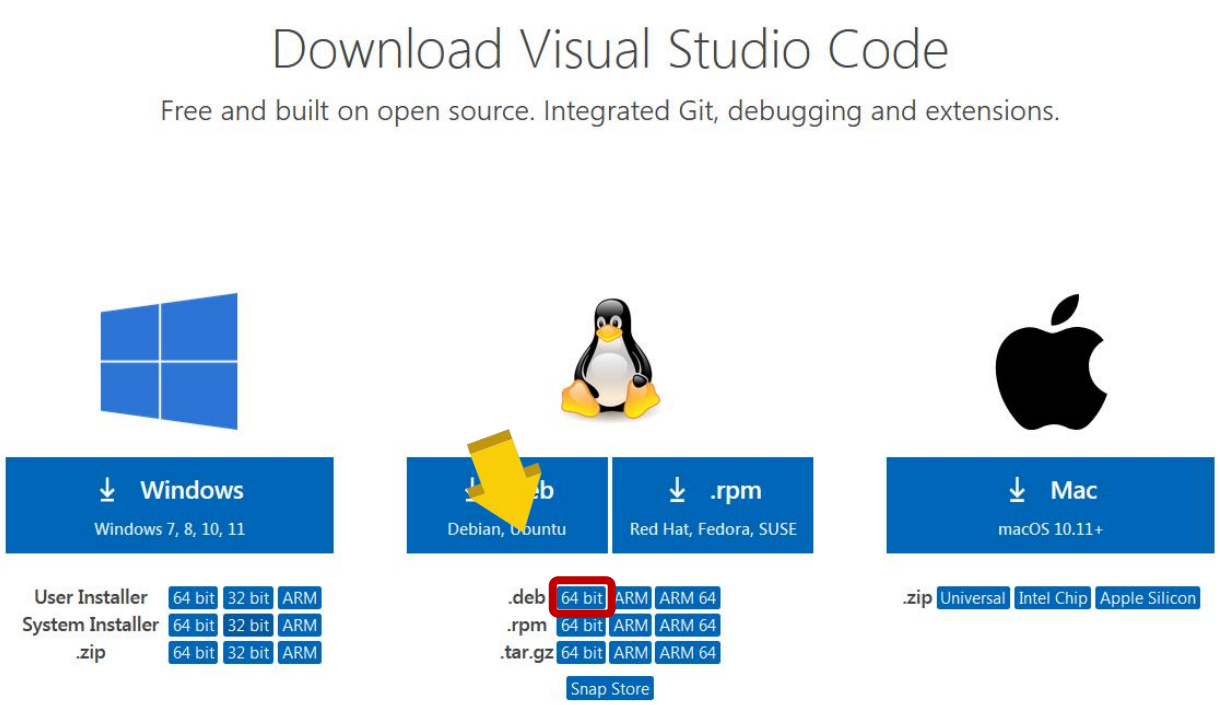
参考讲义：http://www.autolabor.com.cn/book/ROSTutorials/

1.2 VSCode Installation

1.2.1 Download

https://code.visualstudio.com/docs?start=true





1.2 VSCode Installation

1.2.2 Installation

- 安装：
  - 方式1：双击安装即可(或右击选择安装)
  - 方式2：

```
$ sudo dpkg -i xxxx.deb
```

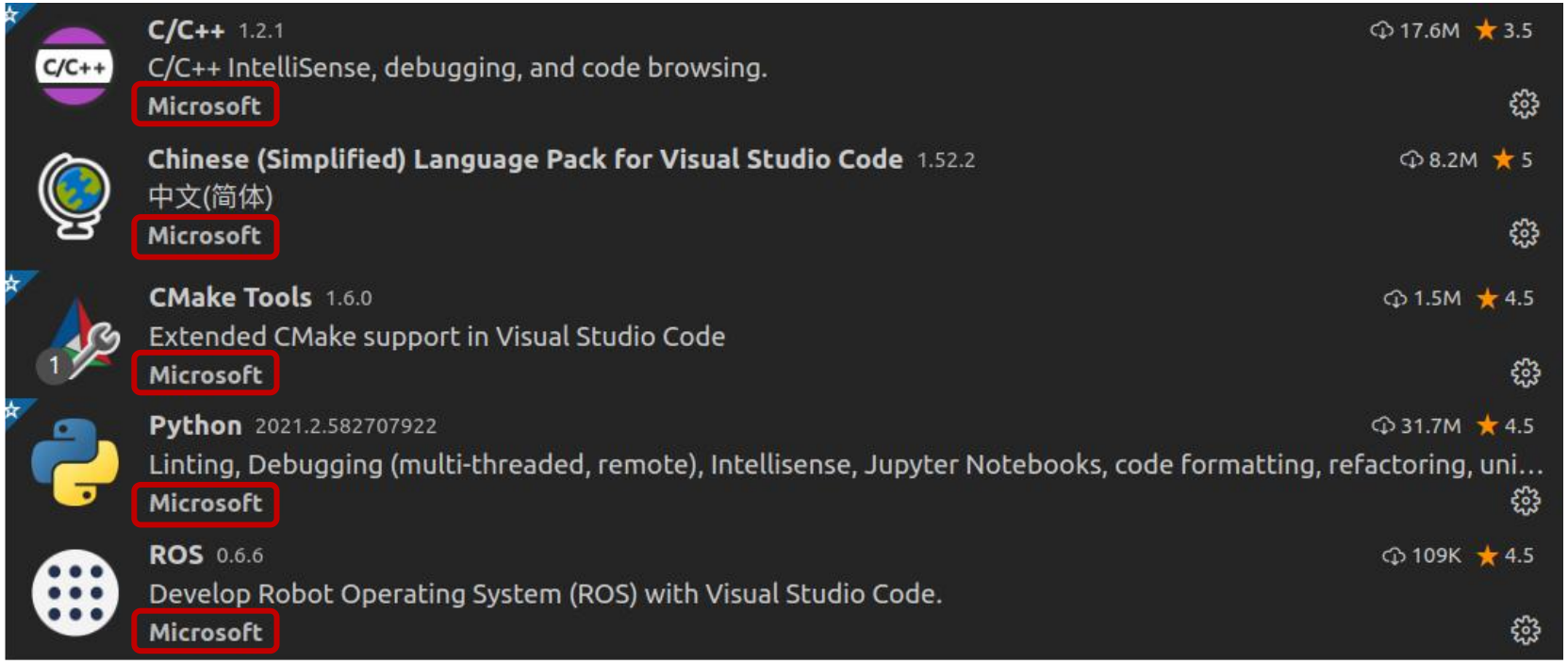
- 卸载

```
sudo dpkg --purge  code
```

1.3 VSCode Application

1.3.1 Plugins

1. C/C++
2. CMake Tools
3. Python
4. ROS
5. Chinese



1.3 VSCode Application

1.3.2 Setups

1. 创建 ROS 工作空间：

```
$ mkdir -p pub_sub_ws/src
```

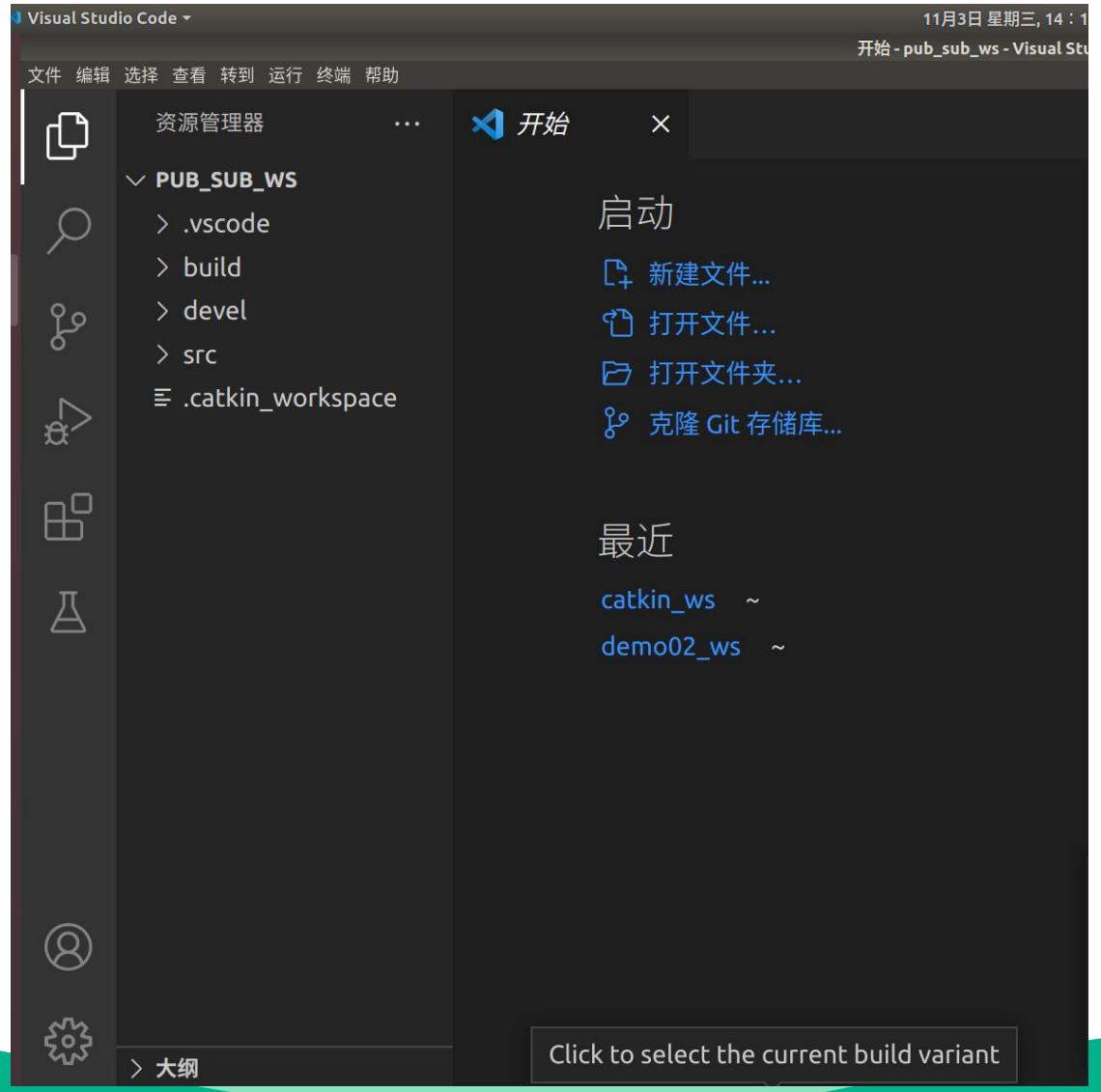
```
$ cd pub_sub_ws
```

```
$ catkin_make
```

2. 启动 VSCode：进入pub_sub_ws 启动 VSCode

```
$ cd pub_sub_ws
```

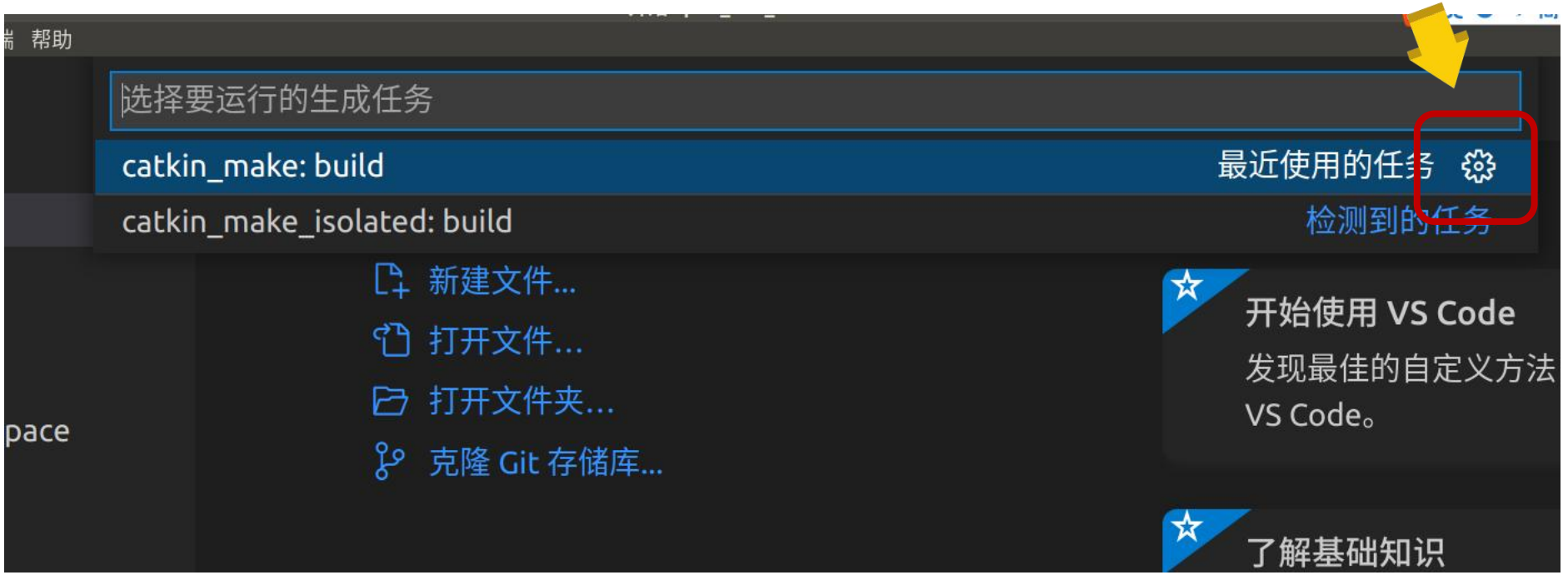
```
$ code .
```



1.3 VSCode Application

1.3.2 Setups

3. 配置 ROS 编译：ctrl + shift + b



1.3 VSCode Application

1.3.2 Setups

3. 配置ROS编译：修改 .vscode/tasks.json 文件，将默认配置替换为以下内容：

```
{
    "version": "2.0.0",
    "tasks": [
        {
            "label": "catkin_make:debug",
            "type": "shell",
            "command": "catkin_make",
            "args": [],
            "group": {"kind":"build","isDefault":true},
            "presentation": {
                "reveal": "always"
            },
            "problemMatcher": "$msCompile"
        }
    ]
}
```

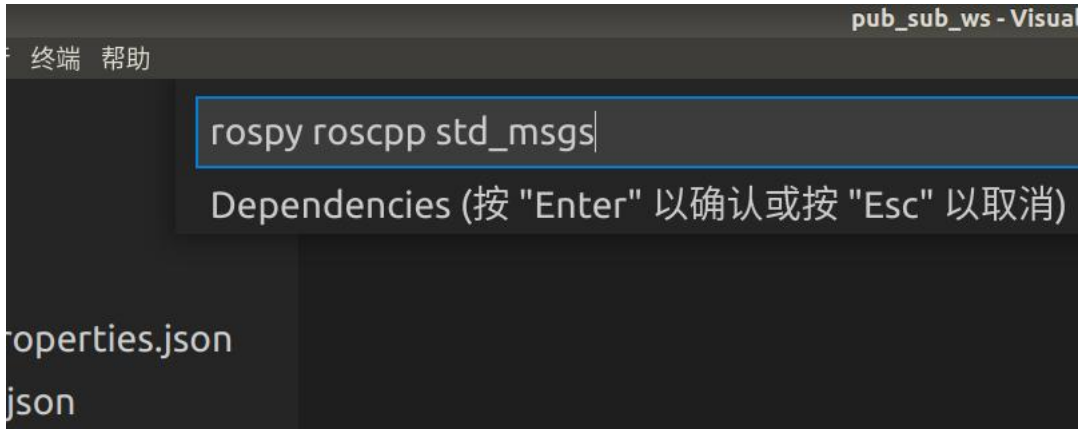
1.3 VSCode Application

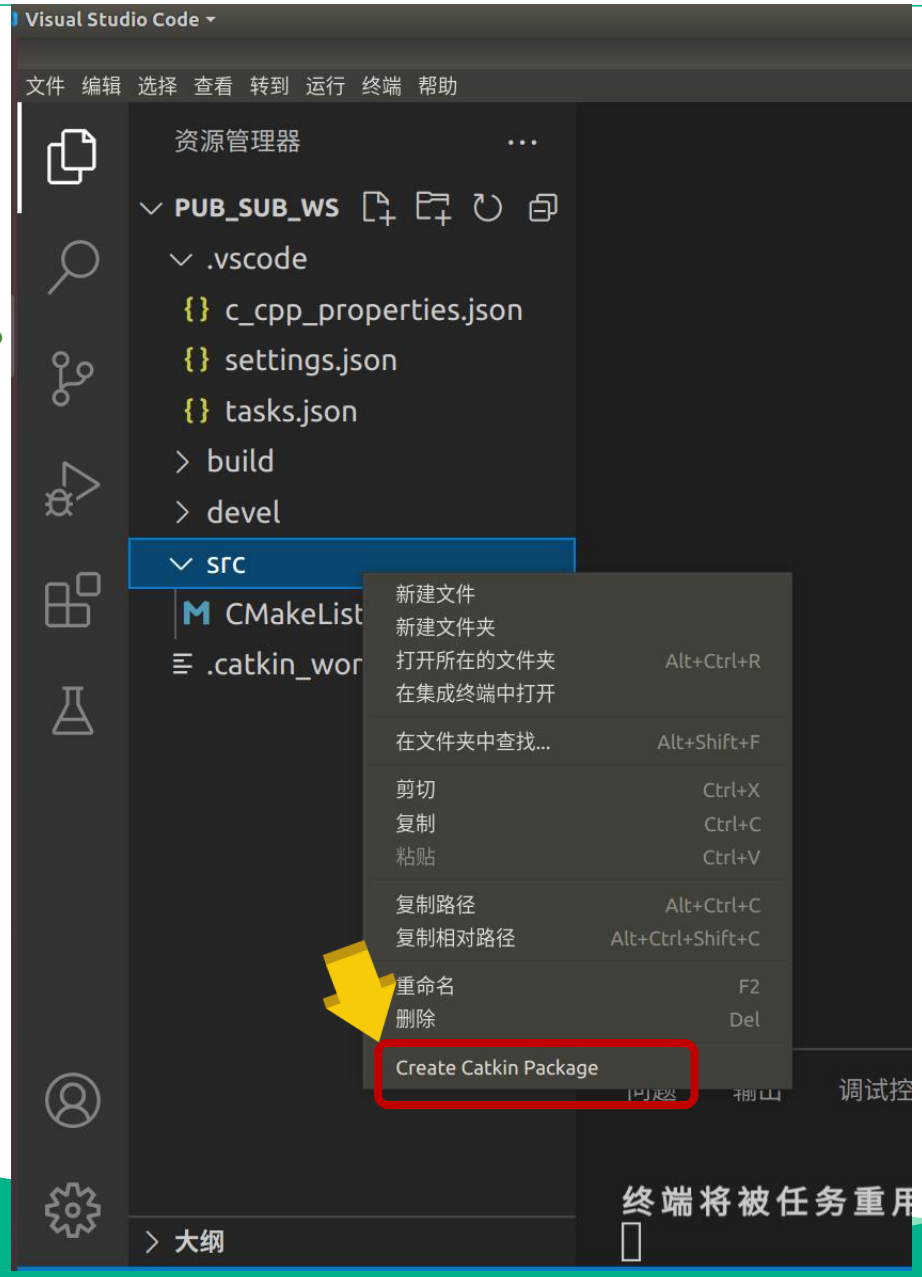
1.3.2 Setups

4. 创建 ROS 功能包

1) 选定 src 右击 ---> create catkin package

2) 设置包名 添加依赖





5. 执行编译：ctrl + shift + b



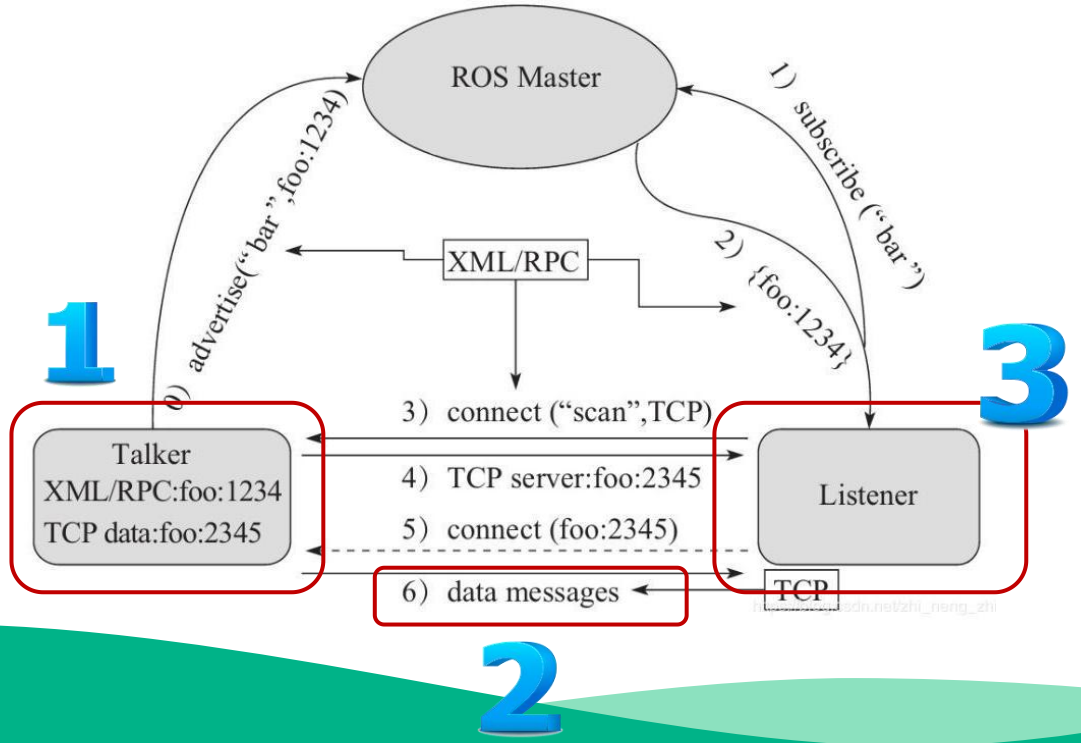
2

Topic Model C++ Code

2.1 Publish node

2.1.1 C++ code

- **需求**：编写发布订阅实现，要求发布方以5HZ(每秒5次)的频率发布文本消息Hello 你好!+数字(数字按频率递增，例如Hello 你好!0、Hello 你好!1、...)，订阅方订阅消息并将消息内容打印输出
- **分析**：在模型实现中，ROS master 不需要实现，连接已经被封装了，需要关注的关键点有三个：
  - 发布方
  - 订阅方
  - 数据(此处为普通文本)
- **整体流程**：
  1. 编写发布方实现
  2. 编写订阅方实现
  3. 编辑配置文件
  4. 编译并执行



2.1 Publish node

2.1.1 C++ code

- **代码实现流程：**

1. 包含头文件
2. 初始化 ROS 节点：命名(唯一)
3. 实例化 ROS 句柄
4. 实例化 发布者 对象
5. 组织被发布的数据，并编写逻辑发布数据

注：附注解的代码在 pub_c.cpp 文件中

2.1 Publish node

2.1.1 C++ code

- 在 pub_sub_test 功能包的 src 文件夹中新建 pub_c.cpp 文件，输入以下代码：

```
#include "ros/ros.h"
#include "std_msgs/String.h"
#include <sstream>

int main(int argc, char  *argv[])
{
    setlocale(LC_ALL,"");
    ros::init(argc,argv,"talker");
    ros::NodeHandle nh;
    ros::Publisher pub = nh.advertise<std_msgs::String>("chatter",10);
    std_msgs::String msg;
    int count = 0;
    ros::Rate rate(1);
```

2.1 Publish node

2.1.1 C++ code

- 在 pub_sub_test 功能包的 src 文件夹中新建 pub_c.cpp 文件，输入以下代码：

```
    while (ros::ok())
    {
        std::stringstream ss;
        ss << "Hello 你好!  Num: " << count;
        msg.data = ss.str();
        pub.publish(msg);
        ROS_INFO("发送的消息:%s",msg.data.c_str());

        rate.sleep();
        count++;
        ros::spinOnce();
    }
    return 0;
}
```

2.1 Publish node

2.1.1 C++ code

- 代码讲解：

```
// 1.包含头文件
#include "ros/ros.h"
#include "std_msgs/String.h" //普通文本类型的消息
#include <sstream>


// 编写mian函数，在 main 函数里面编写代码

int main(int argc, char  *argv[])
{

}
```

2.1 Publish node

2.1.1 C++ code

```
//设置编码，解决中文乱码问题
setlocale(LC_ALL,"");

//2.初始化 ROS 节点：命名(唯一)
        // 参数1 和 参数2 后期为节点传值会使用
        // 参数3 是节点名称，是一个标识符，需要保证运行后，在 ROS 网络拓扑中唯一
ros::init(argc,argv,"talker");

//3.实例化 ROS 句柄，该类封装了 ROS 中的一些常用功能
ros::NodeHandle nh;
```

2.1 Publish node

2.1.1 C++ code

```
//4.实例化 发布者 对象
        //泛型：发布的消息类型，此处为<std_msgs::String>
        //参数1：要发布到的话题，此处为"chatter"
        //参数2：队列中最大保存的消息数，超出此阀值时，先进的先销毁(时间早的先销毁)
ros::Publisher pub = nh.advertise<std_msgs::String>("chatter",10);


//5.组织被发布的数据，并编写逻辑发布数据
        //数据(动态组织)
std_msgs::String msg;

//消息计数器，字符串拼接使用
 int count = 0;
```

2.1 Publish node

2.1.1 C++ code

```
//节点不终止就一直执行循环
 while (ros::ok())
 {
    //使用 stringstream 拼接字符串与编号
    std::stringstream ss;
    ss << "Hello 你好！ Num: " << count;
    msg.data = ss.str();
    //发布消息
    pub.publish(msg);
    //加入调试，打印发送的消息
    ROS_INFO("发送的消息:%s",msg.data.c_str());
    //根据前面制定的发送贫频率自动休眠 休眠时间 = 1/频率；
    rate.sleep();
    count++; //循环结束前，让 count 自增
    //暂无应用
    ros::spinOnce();
 }
```

2.1 Publish node

2.1.1 C++ code

- 修改 CMakeLists.txt：

```
add_executable(sub_c src/sub_c.cpp)

add_dependencies(sub_c ${${PROJECT_NAME}_EXPORTED_TARGETS}
${catkin_EXPORTED_TARGETS})

target_link_libraries(sub_c
  ${catkin_LIBRARIES}
)
```

2.2 Subscribe node

2.2.1 C++ code

- **代码实现流程：**

  1.包含头文件

  2.初始化 ROS 节点:命名(唯一)

  3.实例化 ROS 句柄

  4.实例化 订阅者 对象

  5.处理订阅的消息(回调函数)

  6.设置循环调用回调函数

注：附注解的代码在 sub_c.cpp 文件中

2.2 Subscribe node

2.2.1 C++ code

- 在 pub_sub_test 功能包的 src 文件夹中新建 sub_c.cpp 文件，输入以下代码：

```
#include "ros/ros.h"
#include "std_msgs/String.h"

void doMsg(const std_msgs::String::ConstPtr& msg_p){
    ROS_INFO("接收的消息:%s",msg_p->data.c_str());
}
```

2.2 Subscribe node

2.2.1 C++ code

- 在 pub_sub_test 功能包的 src 文件夹中新建 sub_c.cpp 文件，输入以下代码：

```
int main(int argc, char  *argv[])
{
    setlocale(LC_ALL,"");
    ros::init(argc,argv,"listener");
    ros::NodeHandle nh;

    ros::Subscriber sub = nh.subscribe<std_msgs::String>("chatter",10,doMsg);

    ros::spin();

    return 0;
}
```

2.2 Subscribe node

2.2.1 C++ code

- 代码讲解：

```
// 1.包含头文件
#include "ros/ros.h"
#include "std_msgs/String.h"


// 2. 编写回调函数
void doMsg(const std_msgs::String::ConstPtr& msg_p){
    ROS_INFO("接收的消息:%s",msg_p->data.c_str());
```

1、订阅者代码中，当收到订阅的消息时，激活回调函数，执行回调函数的内容

2、订阅者代码中，要使用 spin() 函数循环读取接收的数据，此过程会阻塞

2.2 Subscribe node

2.2.1 C++ code

- 代码讲解：

```
// 3.编写mian函数，在 main 函数里面编写代码
int main(int argc, char  *argv[])
{

}

//设置编码，解决中文乱码问题
setlocale(LC_ALL,"");

//初始化 ROS 节点:命名(唯一)
ros::init(argc,argv,"listener");

//实例化 ROS 句柄
ros::NodeHandle nh;
```

2.2 Subscribe node

2.2.1 C++ code

- 代码讲解：

```
//实例化 订阅者 对象
ros::Subscriber sub = nh.subscribe<std_msgs::String>("chatter",10,doMsg);

//处理订阅的消息(见回调函数)


//设置循环调用回调函数
ros::spin();//循环读取接收的数据，并调用回调函数处理
```

2.1 Publish node

2.1.1 C++ code

- 修改 CMakeLists.txt：

```
add_executable(pub_c src/pub_c.cpp)

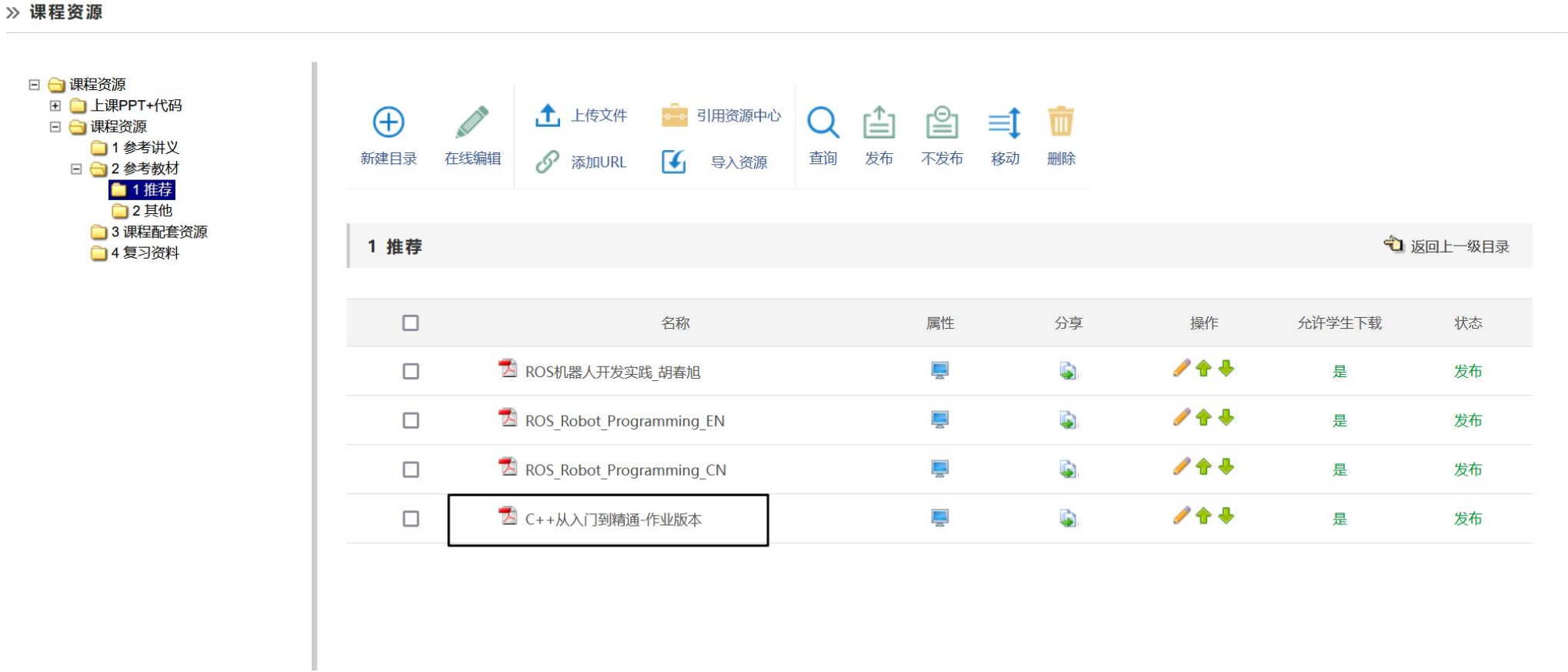
add_dependencies(pub_c ${${PROJECT_NAME}_EXPORTED_TARGETS}
${catkin_EXPORTED_TARGETS})

target_link_libraries(pub_c
  ${catkin_LIBRARIES}
)
```

Summary

1. VSCode
    a) Download
    b) Installation
    c) Plugins
    d) Setups
2. Communication Model 
    a) Topic model
    b) Service model
3. Topic Model C++ Code 
    a) Publush node
    b) Subscribe node

Homework



作业：学习《C++从入门到精通-作业版本》第11章

Q&A

今日事今日毕

Question?